Performance Optimization, Testing, and Release Verification

Measured Baseline for the v0.1.0 Release

SALINLAHI: A Voice Interactive Filipino Language Learning Game

Systems Fundamentals: Virtualization, Containers, and Cloud Computing

Group Number: 33

Group Members:

1. Daquis, Jhesza Mhei G.
2. Lacida, Kylo Bryan
3. Manzanero, Kyla Samantha

Course: BSIT Capstone, University of Perpetual Help System Laguna, JONELTA

Version Measured: v0.1.0 (Android versionCode 2), the release produced in Activity 8

Measurement Date: night of 12 to 13 November 2026

Submitted by: Lacida, Kylo Bryan

Introduction

This activity asks for measurement rather than assumption, so the first thing worth stating is what was actually measured and what was not.

Three measurement sets were produced, all on real code and a real server:

1. The backend the game talks to, timed call by call and split into DNS, TCP, TLS and server time, over the same project, the same endpoints and the same publishable key that ship inside v0.1.0.
2. The pronunciation judgement, which decides right and wrong on every voice attempt, compiled straight out of `Assets/Scripts` and timed per call with its allocations counted.
3. The audio footprint, read from the files themselves rather than estimated, because audio is this project's largest asset class by a wide margin.

One set was **not** produced: frames per second, startup time, scene loading time and resident memory on an Android phone. The member who carried out this activity works on a Windows machine with no Android build support and no phone attached, and the project's own Activity 8 finding stands: the Unity Editor cannot represent this game, because the speech plugin does not exist on desktop and the editor substitutes a keypress for speaking. Rather than present an editor number as a device number, those metrics are named, assigned and scheduled in the monitoring plan in Part IV.

What makes the measurements that were taken worth something is traceability. The released APK was built from commit `c035c8f`. Every source file behind the numbers below is byte identical between that commit and the current `main` at `921c894`:

```
git diff --stat c035c8f..HEAD -- Assets/Scripts/PronunciationChecker.cs \
    Assets/Scripts/WordVariants.cs Assets/Scripts/LeaderboardClient.cs \
    Assets/Scripts/SupabaseClient.cs supabase/
(no output: the files are identical)
```

So these are measurements of the released build's behaviour, not of something that merely resembles it.

Systems Fundamentals · Activity 10

Part I. Performance Context and Resource Applicability

Environment

Item	Value
Project Title	SALINLAHI: A Voice Interactive Filipino Language Learning Game
Current Version	0.1.0, Android versionCode 2, tag `v0.1.0`, commit `c035c8f`
Target Platform	Android 8.0 and above (API 26), `arm64-v8a`
Application Type	Game (mobile), with a cloud backend and a cloud speech recogniser
Test Device / Computer	Lacida's development PC, MSI `MS-7C02` desktop (the measuring machine, not a target device)
Operating System	Windows 10 Pro 64 bit, build 19045
Processor	AMD Ryzen 5 3600, 6 cores and 12 threads, 3600 MHz
Memory / RAM	16.0 GB, 6.0 GB free at measurement time
Network Connection	TP Link wireless adapter, 802.11, 433.3 Mbps link, measured round trip to the Supabase host 43 ms
Backend Under Test	Supabase project `hkfxnifxkcfgelkgvnat.supabase.co`, PostgREST plus one Deno Edge Function
Profiling / Monitoring Tool Used	`Tools/measure_backend.py` (written for this activity: phase split timing over raw sockets), `Tools/check_backend.py` (functional checks over the same path), `Tools/perf_probe` (a .NET 8 harness that compiles the shipping checker sources and times them with `Stopwatch` and `GC.GetAllocatedBytesForCurrentThread`), `curl` phase timing as a cross check, Python `wave` for the audio footprint, PowerShell and Task Manager for the host

Expected behaviour

Performance Area	Expected Behavior	Why It Matters
Startup / Loading	The main menu appears without the child thinking the app is broken, and a chapter scene opens in roughly a second	The users are Kinder to Grade 3 pupils. A child who is not shown something quickly taps again, which the game reads as a second input
Runtime Responsiveness	Judging a spoken word is instant relative to the speaking itself, and the character reacts in the same breath	Speaking is the only control in this game. A delay between saying *Bangon* and Kylo getting up breaks the one connection the whole design rests on
Memory Usage	Stable through a full chapter, with no growth across repeated scenes, on a low specification phone	Classroom tablets are not new. An app that grows until it is killed loses the child's progress in the middle of a lesson
Save / Load	A profile saves on finishing a scene and survives a restart, without a visible pause	Progress is the reward structure. A save that is slow enough to notice, or lost, undoes the session
Network Response	The board appears within about a second of tapping LEADERBOARDS; a score submission never blocks play; being offline is handled rather than shown as an error	The leaderboard is the only shared feature, and the recogniser is itself an online service, so the network is on the critical path twice

Resource applicability

Resource	Applicable?	Where It Affects the System	Possible Concern
CPU	Partial	`PronunciationChecker.IsCorrect` runs on the main thread for every recogniser result; `PoseSequence` drives drawn animation frames in the Paaralan scenes; `VoiceStatusMachine` and the HUD update per frame	The judgement does string normalisation and regular expression work per attempt. Attempts are a few per second at most, so the exposure is small, but it is on the thread that draws the frame
Memory	Yes	Five music tracks of 177 seconds each, imported as Decompress On Load; 423 narration clips loaded on demand; sprite sheets for the drawn character animation	A decompressed music track is 29.8 MB of PCM resident while it plays. That is the largest single allocation in the game and it is settings driven, not code driven
Disk / Storage	Yes	A 151.3 MB package; `players.json` written on scene completion; `salinlahi_voice_attempts.csv` appended once per spoken word; 423 narration clips read on demand	The attempt log grows without bound, and the package is close to the size at which a classroom install over a slow connection becomes a problem
Network	Yes	The leaderboard read, the rank call, the nickname check, the score submission through the Edge Function, and the Filipino recogniser itself, which `VoiceCommand` calls with `preferOfflineRecognition: false`	Every spoken word is a network round trip to Google's recognition service. This is the heaviest network path in the game and the only one that could not be measured from the development machine

No area is marked Not Applicable. The network row deserves the note that this project genuinely cannot mark it so: without a connection the game cannot hear a single word.

Part II. Functional Validation and Performance Baseline

System scenario selected: retrieve leaderboard records. A child taps LEADERBOARDS on the main menu, the board is read from Supabase, the child's own rank is resolved if they are outside the fetched page, and six rows are drawn.

It was chosen because it is important (the only shared feature in the system), repeatable (a read, with no side effect), and measurable end to end from a machine that is not the target device, since the slow part of it is the server and the network rather than the phone.

A second, narrower scenario is included alongside it: judging one spoken word. That one cannot be exercised on a phone from here, but the component that decides the answer is plain C# and was measured directly.

A. Functional validation

Nine checks were run against the live project with the key the game ships. Seven are reads. Two are writes that Row Level Security must refuse, and for those the table row itself is read before and after, so a pass is proved by the data being unchanged rather than only by a status code.

Evidence: `Docs/perf/20261113_backend_functional_checks.txt`, reproduced with `python Tools/check_backend.py`.

Requirement / Feature	Test Action / Input	Expected Result	Actual Result	Status
The board can be read by the shipped build	`GET /rest/v1/leaderboard?select=...&order=total_points.desc,total_stars.desc,updated_at.asc&limit=10` with the `sb_publishable_` key	HTTP 200 and a JSON array	HTTP 200, 1 row, 154 bytes	Pass
Only what the panel draws is returned	Inspect the keys of every returned row	The five display columns and nothing else	No other column present	Pass

Requirement / Feature	Test Action / Input	Expected Result	Actual Result	Status
The database does the ordering, not the client	Compare the returned order against points then stars	Rows already in board order	Rows in order as returned	Pass
A player's rank can be resolved	`POST /rest/v1/rpc/leaderboard_rank` with the top row's `player_id`	One row, rank 1	HTTP 200, rank 1	Pass
An unknown player does not produce a wrong rank	Same RPC with `00000000-0000-0000-0000-000000000000`	An empty result, which the client reads as "no row yet"	HTTP 200, body `[]`	Pass
The recent scores read is ordered newest first	`GET /rest/v1/player_scores?select=*&order=created_at.desc&limit=20`	20 rows, timestamps descending	HTTP 200, 20 rows, 4,020 bytes, in order	Pass
The game's key cannot insert a leaderboard row	`POST /rest/v1/leaderboard` with a forged row worth 999,999 points	Refused, and the table unchanged	HTTP 401, 1 row before and 1 row after	Pass
The game's key cannot change a leaderboard row	`PATCH /rest/v1/leaderboard?player_id=eq.<top row>` setting 999,999 points	Refused, and the row unchanged	HTTP 204, and the top row's points unchanged	Pass
No token hash is readable with the game's key	`GET /rest/v1/leaderboard_owner?select=*`	Refused or empty	HTTP 401, 0 rows	Pass

One of those rows is worth pausing on. The refused update answered **HTTP 204**, not 401. PostgREST runs the statement, Row Level Security gives it no rows to touch, and the update succeeds at changing nothing. A check written against the status code alone would have recorded a successful forged write as a pass. The check was corrected to assert on the row, and the row is unchanged. That is precisely the trap this activity warns about: an artifact responding is not the same as a feature working.

The judgement component was validated separately, with the shipping sources compiled directly. Evidence: `Docs/perf/20261113_voice_judgement_baseline.txt`, reproduced with the .NET SDK inside the Unity install running `Tools/perf_probe`.

Requirement / Feature	Test Action / Input	Expected Result	Actual Result	Status
A correctly said word is accepted	`IsCorrect("bangon", "Bangon")`	Accepted	Accepted	Pass
Recogniser filler words do not break a correct answer	`IsCorrect("Kuha mo", "Kuha")`	Accepted	Accepted	Pass
A different taught word is not accepted	`IsCorrect("kanin", "Kain")` and `IsCorrect("sapatas", "Sapatos")`	Refused	Refused	Pass
Silence and unrelated speech are not answers	`IsCorrect("", "Kuha")` and `IsCorrect("mali", "Kuha")`	Refused	Refused	Pass
Every taught word is accepted when said exactly	All 53 words in `WordVariants` against themselves	53 accepted	53 accepted	Pass
No taught word is accepted for another	All 2,756 ordered pairs of distinct taught words	None accepted	None accepted	Pass

The defect this validation does not clear

Activity 8's device test found that a deliberately mispronounced word was accepted during play on a phone. Nothing above contradicts that, and nothing above closes it. The 2,756 pair sweep says the checker refuses a wrong word every time it is asked; the device says the child was still marked correct. Both can be true if the checker is never asked the real question, which is the standing hypothesis: the device's Filipino dictation service returns the nearest real Filipino word rather than the sounds actually made, so the error is repaired before the checker sees it.

That defect is a correctness defect, it is already documented as the most serious known issue in `Docs/release/v0.1.0/RELEASE_NOTES.txt`, and it is the subject of the hotfix worked example in Part VI. It is recorded here, before any performance work, because this activity requires a failing functional test to be documented rather than stepped over.

B. Performance baseline

Twenty runs per metric. Every cold row opens a fresh connection, which is what the game does when the child taps LEADERBOARDS for the first time after the app starts. The warm row is a second request on a connection already open, so it shows the cost of an extra call without an extra handshake.

Evidence: `Docs/perf/20261113_backend_baseline.csv`, reproduced with `python Tools/measure_backend.py 20`.

Metric	Test Scenario	Baseline Result	Tool Used
Board fetch, total time	Open the board on a fresh connection, 20 runs	median **226.6 ms**, min 199.8, mean 231.4, max 312.2	`Tools/measure_backend.py`
Board fetch, phase split	Same 20 runs, median of each phase	DNS 0.3 ms, TCP 43.2 ms, TLS 85.5 ms, request and response 91.1 ms	`Tools/measure_backend.py`
Rank call on an open connection	The second request the board makes when the child is outside the fetched page	median **83.8 ms**, min 61.6, max 120.1	`Tools/measure_backend.py`
Board payload	Bytes returned for the board, current data	154 bytes for 1 row, about 150 bytes per row	`Tools/measure_backend.py`
Recent scores read	`select=*` with `limit=20` on `player_scores`	median 214.4 ms, 4,020 bytes for 20 rows	`Tools/measure_backend.py`
Nickname check	`ilike` lookup with `limit=200`, as the OKAY button issues it	median 216.1 ms, 24 bytes	`Tools/measure_backend.py`
Score submission, refusal path	`POST` to the Edge Function with an invalid `player_id`, refused before any database access	median 223.1 ms, HTTP 400	`Tools/measure_backend.py`
Voice judgement, per attempt	One recogniser result judged against one live target, 200,000 calls in 10 batches	median **5.64 us** per attempt, min 2.86, max 9.19	`Tools/perf_probe`
Voice judgement, first call	The call that pays static initialisation and regular expression construction	**15.6 ms**, once per application run	`Tools/perf_probe`
Voice judgement, allocation	Bytes allocated per attempt, and collections over the batch	682 bytes per attempt, 18 generation 0 collections per 200,000 attempts	`Tools/perf_probe`
Music memory cost	PCM resident per playing track, from the file's own header	29.8 MB per track for the five 177 second tracks	Python `wave`, `Docs/perf/20261113_audio_footprint.txt`

Metric	Test Scenario	Baseline Result	Tool Used
Audio on disk	Source audio shipped with the project	423 narration clips, 124.7 MB, median clip 313 KB; 6 music files, 150.6 MB	Python `wave`
Data volume at measurement	Rows behind the two reads	`leaderboard` 1 row, `player_scores` 67 rows	PostgREST `count=exact`

Conditions held constant across every run: the same machine, the same wireless connection, the same Supabase project, the same source commit, the same queries, and twenty runs per metric taken within the same few minutes.

A second tool was used as a cross check rather than trusting one implementation. `curl` with phase timing, issued directly, returned 227 ms to 278 ms over five runs of the board fetch, against the 199.8 ms to 312.2 ms range above. Two independent clients agreeing within a few tens of milliseconds is the reason the figure is quoted as roughly a quarter of a second rather than to the millisecond.

Part III. Bottleneck Analysis and Performance Decision

Primary Area Reviewed: Network, with Memory reviewed second.

Observed Behavior. Opening the board costs about 0.23 seconds, and most of that is not the query. The TCP handshake is 43 ms and the TLS handshake is 85 ms, so 129 ms of the 227 ms is spent getting a connection before a single byte of the request is sent. The round trip to the server is itself 43 ms, so of the 91 ms spent on the request and response, roughly 48 ms is Supabase doing the work and the rest is distance. The same shape holds for every other call: all four cold measurements land between 214 ms and 227 ms regardless of whether they return 24 bytes or 4,020.

Supporting Evidence. The phase split in `Docs/perf/20261113_backend_baseline.csv`, and the warm row: the same server, asked a second question on a connection already open, answers in 83.8 ms rather than 226.6 ms.

Possible Cause. Connection setup, not application code. Each `UnityWebRequest` in `LeaderboardClient` and `SupabaseClient` is created, used once and disposed, and the project's Supabase instance is in another region. Neither of those is a defect; both are the price of a hosted database that a free tier project has no reason to pay differently.

Effect on User Experience. The board shows "Loading..." for about a quarter of a second, and about a third of a second when the child is outside the fetched page and the rank call follows. Nothing blocks play: the submission path is asynchronous by design and an offline device is handled with a written message rather than an error.

Observation	Supporting Evidence	Possible Cause	Priority
Every backend call costs about 0.22 s, and 57 percent of it is connection setup rather than query time	Board fetch median 226.6 ms, of which TCP 43.2 ms and TLS 85.5 ms; the same call on an open connection costs 83.8 ms	One fresh TCP and TLS handshake per `UnityWebRequest`, plus 43 ms of distance to the region	Low
Opening the board can cost two round trips rather than one	`LeaderboardClient.FetchRoutine` calls `RankRoutine` when the player is not in the fetched ten; measured at 83.8 ms warm	The rank is deliberately asked of the database instead of pulling the whole board down to count it	Low
`player_scores` has no index on `created_at` or `player_name`, and the recent scores read uses `select=*`	`supabase/migrations/20261112_leaderboard.sql` indexes only `leaderboard`; the read returns 4,020 bytes for 20 rows and takes the same 214 ms as a 24 byte reply	Both queries are sequential scans whose cost grows with the row count, currently hidden because the table holds 67 rows	Medium, as a growth risk only

Observation	Supporting Evidence	Possible Cause	Priority
A playing music track holds 29.8 MB of PCM in memory	`Docs/perf/20261113_audio_footprint.txt`; five tracks of 177 seconds at 44.1 kHz stereo, imported with `loadType: 0`, which is Decompress On Load	An import setting on five assets, not code. Streaming would reduce it to a buffer	Medium, unconfirmed on a device
The first voice attempt of a run costs 15.6 ms, every later one 5.64 us	`Tools/perf_probe`, cold call measured separately from the warm batches	Static initialisation of the filler word set and construction of the regular expressions, paid once	Low
Frames per second, startup time, scene load time and resident memory on a phone are unmeasured	No Android toolchain and no device on the measuring machine; the editor cannot exercise the voice mechanic at all	Measurement gap, not a performance finding	Medium, and scheduled in Part IV

Decision

B. Optimization Is Not Currently Necessary.

The reasoning, against the evidence rather than against expectation:

1. The scenario measured completes in about a quarter of a second, and 57 percent of that is handshaking and distance. No change to this project's code can remove a TLS handshake or move a region, and the remaining 48 ms of actual server work is not worth attacking.
2. The payloads are tiny. The largest reply in the whole system is 4,020 bytes. There is no transfer problem to solve.
3. The per attempt CPU cost of the judgement is 5.64 microseconds with 682 bytes allocated. At a realistic few attempts per second that is under 2 KB per second and far below one percent of a frame. Optimising it would be work with no observable effect.
4. The two genuine growth risks, the unindexed columns and the music import settings, are growth risks rather than present symptoms. The unindexed columns cannot be addressed from the client at all: adding an index is a `create index` statement that needs dashboard access, which the publishable key does not have.
5. The system's open defect is a correctness defect, not a performance one. The honest ranking of this project's problems puts "a mispronounced word is accepted" far above "the board takes a quarter of a second", and spending this cycle on the second would be optimising the wrong thing while the first is open.

The activity's own instruction is the one followed here: do not change working code only to produce an optimization result.

Part IV. Performance Monitoring Plan

Option B was selected, so this is the monitoring plan rather than a change.

Metric / Resource	Current Result	When It May Become a Concern	When to Measure Again
Board fetch, total time	median 226.6 ms, max 312.2 ms	Median above 600 ms, or the server leg above 250 ms, which would mean the delay has moved from distance to the query	Before every release build, with `python Tools/measure_backend.py 20`
Board fetch, server leg	91.1 ms, of which about 43 ms is travel	Rising while TCP and TLS stay flat, which points at the table rather than the network	Same run as above
Requests per board open	1, or 2 when the child is outside the fetched ten	More than 2, or any per frame or per keystroke call appearing on the menu	Before every release build, and after any change to `LeaderboardClient` or `NicknameAvailability`

Metric / Resource	Current Result	When It May Become a Concern	When to Measure Again
Rows in `leaderboard` and `player_scores`	1 and 67	`player_scores` past about 5,000 rows, where the unindexed `created_at` ordering and the `ilike` name lookup stop being free	Monthly, and before any classroom session with more than one device
Recent scores payload	4,020 bytes for 20 rows with `select=*`	Past about 50 KB per read, or when a column larger than `remarks` is added	With the row count check above
Resident memory on a phone	Unmeasured	Any growth across repeated scenes, or audio above about 60 MB	First device session with the Unity Memory Profiler, assigned below
Music memory	29.8 MB of PCM per playing track, by arithmetic from the file header	Confirmed above about 30 MB on a 2 GB device, or an audible stutter at a music change	Same device session, in the same Memory Profiler capture
Startup and scene load time	Unmeasured	Above about 3 seconds to the menu, or 1.5 seconds to a chapter scene	First device session, with the Unity Profiler on a build, not in the editor
Voice recognition round trip	Unmeasured, and it is a network call on every attempt	Above about 1.5 seconds from speaking to a verdict, which is where a child gives up and taps	From the timestamps in `salinlahi_voice_attempts.csv` after the next device session
Voice judgement per attempt	5.64 us, 682 bytes allocated	Above about 1 ms on a device, or allocations rising as `WordVariants` grows	Whenever `PronunciationChecker` or `WordVariants` is edited, with `Tools/perf_probe`
Attempt log size	About 100 bytes per row, so roughly 1 MB per 10,000 attempts	Past about 5 MB on a shared classroom device, where it should be rotated or trimmed	After each classroom session
Package size	151.3 MB	Past about 200 MB, where installing over a classroom connection becomes the slowest part of a deployment	At every build, from `BUILD_INFO.txt`

Why is optimization not currently necessary? Because the measurements show the chosen scenario completing in about a quarter of a second, with more than half of that spent on a TLS handshake and the distance to the region, payloads of at most four kilobytes, and a per attempt judgement cost of under six microseconds. Nothing in the evidence identifies application code as the limit. The two items worth watching are a schema that is unindexed for growth it has not yet seen, and an audio import setting whose cost is real but unconfirmed on a device. Changing either now would be a change made without a symptom and without a way to prove it helped.

Which future feature or project change could affect performance? The Palengke level and the sixth Bahay scene, which add scenes and assets to an already 151 MB package. The remaining 20 narrator recordings, which add to the audio set. Classroom use with several devices, which is the first time the leaderboard will hold more than a handful of rows. Any move of the nickname check from the OKAY button to a per keystroke check, which would turn one request into one per letter typed. And per player filtering and paging on the leaderboard, which is the first query in this system that would benefit from an index that does not exist.

Which metric should continue to be monitored? The board fetch server leg, because it separates our problem from the network's: if TCP and TLS stay at 43 ms and 85 ms while the server leg climbs, the cause is the table or the query and nothing else. Alongside it, resident memory on a device, which is the only area where a real concern is suspected and not yet measured.

When should the group investigate the metric again? Before every release build, as a step in `Docs/RELEASE_PROCESS.md`; after any change to `LeaderboardClient`, `SupabaseClient`, `NicknameAvailability` or the migration; and once `player_scores` passes about 5,000 rows. The device measurements are not scheduled by a threshold but by the next opportunity: the first session where a member has the phone and a build, which is the same session that must gather the pronunciation attempt logs.

Part V. Re profiling and Functional Regression Testing

No optimization was implemented, so this part does what the activity asks in that case: one important measurement and the functional tests were repeated, to establish that the baseline is a baseline and not a single lucky reading.

A. Measurement consistency

The full twenty run measurement was repeated about ten minutes after the first, on the same machine, the same connection and the same commit. Evidence: `Docs/perf/20261113_backend_recheck.csv` and `Docs/perf/20261113_voice_judgement_recheck.txt`.

Metric	First Run	Second Run	Difference	Interpretation
Board fetch, median	226.6 ms	225.0 ms	1.6 ms, 0.7 percent	Stable. The baseline is reproducible
Board fetch, TLS phase	85.5 ms	86.4 ms	0.9 ms	Stable, and consistent with one handshake over the same link
Board fetch, server leg	91.1 ms	90.5 ms	0.6 ms	Stable. No sign of a warming cache or a cold start effect
Rank call on an open connection	83.8 ms	86.3 ms	2.5 ms, 3 percent	Stable, and still about a third of a cold call
Recent scores read, median	214.4 ms	216.0 ms	1.6 ms	Stable, and unchanged in bytes at 4,020
Voice judgement, per attempt	5.64 us	5.19 us	0.45 us, 8 percent	Within the noise of a desktop scheduler. The order of magnitude, microseconds, is the finding
Voice judgement, first call	15.6 ms	13.4 ms	2.2 ms	Varies with the machine's state. Still a once per run cost in the tens of milliseconds
Voice judgement, allocation	682 bytes	682 bytes	0	Identical, as it should be for deterministic code on the same input

The maximum values are the one place the two runs differ meaningfully: the first run saw a single 312 ms board fetch, the second a single 678 ms one. Both are the wireless link, not the server, and both are the reason the plan above watches the median and the server leg rather than the worst case.

B. Functional regression testing

The same checks were re run after all measurement, including after the two deliberate write attempts, to confirm that nothing done during this activity changed the system. Evidence: `Docs/perf/20261113_backend_functional_recheck.txt`.

Feature / Test	Expected Result	Actual Result	Status
Board reads with the shipped key	HTTP 200, rows returned	HTTP 200, 1 row, 154 bytes	Pass
Board returns only the five display columns	No other column	No other column	Pass
Board ordered by the database	Points then stars	In order	Pass
Rank for a player on the board	Rank 1 for the top row	Rank 1	Pass
Rank for an unknown player	Empty, not a wrong number	`[]`	Pass
Recent scores ordered newest first	20 rows, descending	20 rows, descending	Pass
Forged insert still refused	HTTP 401 and the table unchanged	HTTP 401, 1 row before and after	Pass
Forged update still changes nothing	The top row unchanged	Unchanged, still 0 points	Pass
Token hashes still unreadable	Refused or empty	HTTP 401, 0 rows	Pass

Feature / Test	Expected Result	Actual Result	Status
Voice judgement, all six behaviour cases	As in Part II A	All six as expected	Pass
Every taught word accepted when said exactly	53 of 53	53 of 53	Pass
No taught word accepted for another	0 of 2,756 pairs	0 of 2,756 pairs	Pass

The leaderboard holds one row, with the same nickname and the same zero totals, before and after this activity. The two forged writes left no trace, which is what the before and after row counts are there to show.

Part VI. New Build, Version, Patch, and Hotfix

A. When to create another build

The rule the group follows is that a build is produced when an accepted change reaches the application that will be tested or distributed: a bug fixed, a performance improvement made, a security correction applied, a feature added, behaviour or configuration changed, or a release candidate to be tested.

This activity produced no such change. What it produced is three measurement tools, seven evidence files and this report. None of them is compiled into the APK: `Tools/` holds standalone scripts and a .NET console project that the Unity build never sees, and `Docs/` is documentation. Nothing under `Assets/`, `ProjectSettings/` or `supabase/` was touched.

One repository fault was found and fixed while committing this report, and it belongs here because it is a release artifact problem rather than a performance one. Git's built in attributes mark a PDF for plain text diffing but never mark it binary, so with `core.autocrlf` set to true, which is the default on Windows, Git rewrites every LF byte inside a PDF on checkout. The working copy of `Docs/Daquis_Group33_FirstRelease.pdf` on this machine had a different checksum from the blob it came from, which means the Activity 8 report was damaged by being checked out rather than by anything anyone did to it. `.gitattributes` now marks `*.pdf`, `*.docx`, `*.pptx` and `*.xlsx` as binary, and the working copies were restored from their blobs and verified against them. No application file is affected, so this does not change the build answer below.

There is, however, a separate fact worth recording honestly, because it affects the next build rather than this one. `main` has moved ahead of the released build in application code. Since `c035c8f`, commit `0102bbe` changed `Assets/Scripts/SceneObjectiveController.cs` and `Assets/Scripts/WalkIntoScene.cs`, and commit `eca4868` rebuilt `Assets/Scenes/Chapter2_Level1_Paaralan.unity` with the guard's gate animation and new sprites. Those are accepted changes that will require a build when the group next releases. They do not require one today, and they are not what this activity changed.

B. Version number

The convention is unchanged from Activity 8: `MAJOR.MINOR.PATCH`, with development versions below 1.0.0 while the system is incomplete.

Level	Used For	This Project's Next Example
PATCH	A correction with no new capability	0.1.0 to 0.1.1, for the pronunciation acceptance defect
MINOR	A new working feature or meaningful capability	0.1.0 to 0.2.0, for the Paaralan gate animation already on `main`, or for the Palengke level
MAJOR	A major release or an incompatible change	1.0.0, only when every condition in `Docs/RELEASE_PROCESS.md` holds

Which of those comes next depends on which change lands first, and the group should decide that rather than discover it:

1. If the pronunciation correction is made against the released version, it is a PATCH taken off the `v0.1.0` tag and released as **0.1.1**, with the Paaralan animation left on `main` for the release after it.
2. If the group instead builds from `main` as it stands today, the animation changes are already in it, so that build is **0.2.0** and not 0.1.1, because it carries new working behaviour and not only a correction.

Either way the Android version code must rise with it, from 2 to 3. Activity 8 recorded why: a phone refuses to install a new build over an old one when the code has not moved.

C. Normal bug fix

```
Version 0.1.0
      v
Bug found during ordinary development or testing
      v
Fixed on a branch, reviewed, merged
      v
Regression test
      v
Version 0.1.1
      v
New build
```

A normal bug fix is the ordinary case and most of this project's corrections are that: the package identifier and the missing internet permission found during Activity 8's readiness check were both found before anything was distributed, so neither was a hotfix.

D. Hotfix

```
Released Version 0.1.0
      v
Critical issue identified in the released build
      v
Hotfix branch cut from the released source
      v
Focused correction only
      v
Test, then regression test
      v
Version 0.1.1
      v
New build
      v
Smoke test
      v
Corrected release
```

This project has a real candidate, which is why the distinction matters here rather than in the abstract. **A mispronounced word is accepted as correct during play on a device.** It is in a tagged, published release that other people can install, it defeats the purpose of the system rather than annoying the user, and it cannot wait for the Palengke level.

It is not being treated as a hotfix in this activity, and the reason is evidence rather than convenience. The cause is not yet known. The checker refuses a wrong word in all 2,756 pair checks, so changing the checker now would be changing a component that the measurements say is behaving, on a hypothesis nobody has tested. The hypothesis is that the device's dictation service repairs the word before the checker sees it, and the test for it already exists in the build: `VoiceAttemptLog` writes `rawSpokenText` beside `canonicalSpoken` for every attempt. One device session with a deliberately mispronounced word settles it. Until that session happens, a hotfix would be a guess shipped urgently, which is worse than the defect being known and documented.

E. Hotfix procedure

The procedure the group will follow, written against this project rather than in general terms:

1. Identify the affected released version: 0.1.0, tag `v0.1.0`, commit `c035c8f`, Android version code 2.
2. Start from the released source, not from `main`: `git checkout -b hotfix/pronunciation-accepts-wrong-word v0.1.0`.
3. Implement only the correction the attempt logs justify. The Paaralan animation work on `main` does not come along, even though it is finished, because an urgent fix should change as little as possible.
4. Test the corrected feature on a device: say a taught word wrongly on purpose and confirm it is refused, which is the test that failed.
5. Regression test around it: the ten release checks from Activity 8 Part VI, the 2,756 pair sweep through `Tools/perf_probe`, and the nine backend checks through `Tools/check_backend.py`.
6. Raise the version to 0.1.1 and the Android version code to 3 in Player Settings, which is the single source the build reads.
7. Produce the build through the one entry point: `Tools > SALINLAHI > Build Release APK`, from the main checkout, because the Supabase configuration asset is gitignored and exists only there.
8. Smoke test the packaged APK, as in Part VII.
9. Release it: tag `v0.1.1`, attach the zipped package, and update the release notes so the known issue moves from Known Issues to Fixed, with the device evidence that justifies the move.
10. Merge the hotfix branch back into `main`, so the correction is not lost the next time a build is made from the development line.

F. Build decision record

Item	Response
Was application code/configuration changed?	**No.** Nothing under `Assets/`, `ProjectSettings/` or `supabase/` was modified. The additions are `Tools/measure_backend.py`, `Tools/check_backend.py`, `Tools/perf_probe/`, seven evidence files under `Docs/perf/`, three `.gitignore` entries covering the probe's build output and its project file, four `.gitattributes` entries marking documents binary, this report, and the documentation that points at them: a checks section in `Docs/RELEASE_PROCESS.md` and two additions to `README.md`
Is another build required?	**No**, not by this activity
Previous Version	0.1.0 (Android version code 2)
New Version, if applicable	None produced. The next build will be 0.1.1 if it is the pronunciation correction taken off the `v0.1.0` tag, or 0.2.0 if it is built from `main`, which already carries the Paaralan animation
Change Type	Other: measurement, functional verification and documentation
Source Reference	Measured against `main` at `921c894`, whose measured source files are byte identical to the released commit `c035c8f`, tag `v0.1.0`. This activity's own work lands in one commit on top of `921c894`
Reason for Version Change	None. The version identifies the application, and the application did not change. Raising it for a documentation commit would break the one thing the number is for, which is telling a tester exactly what they are holding

Part VII. Build Verification and Smoke Test

No new build was produced because no application change requiring another build was made during this activity.

Two further facts belong beside that sentence rather than hidden behind it. The measuring machine has Unity 6000.6.2f1 but only Windows standalone build support, with no Android module and no device attached, so an Android build could not have been produced here in any case; the Android toolchain is on the member who

performed the Activity 8 release. And a Windows build was deliberately not made as a substitute, because the speech plugin does not exist on desktop, so a desktop package cannot smoke test the only feature that matters.

The smoke test below is therefore written and not run. It is the test to execute on the next build, whether that build is 0.1.1 or 0.2.0, and it is recorded now so that the next person does not have to invent it under time pressure.

Smoke Test	Expected Result	Actual Result	Status
Application launches	The main menu opens on the device	Not yet run	Pending the next build
Main menu / entry point	SETTINGS then ABOUT shows the new version number, and a level can be entered	Not yet run	Pending the next build
Critical feature	A taught word said correctly is accepted, and the same word said wrongly is refused	Not yet run	Pending the next build
Save/load and integration	A profile survives a restart, and a finished scene's total reaches the leaderboard	Not yet run	Pending the next build
Application closes correctly	The app exits without hanging	Not yet run	Pending the next build

Part VIII. Performance and Release Monitoring

A checklist the group reuses on every succeeding build, folded into the steps already in `Docs/RELEASE_PROCESS.md`.

Check	Expected Condition
Important functional tests	Pass, including the nine backend checks and the voice judgement checks, run from `Tools/` rather than by hand
Critical known defects	Resolved, or written into the release notes in the words a user would need
Performance	No unexplained regression: board fetch median within about 150 ms of 226 ms on a comparable connection, and the server leg under 250 ms
Memory	No continuous growth across repeated scenes on a device, and audio under about 60 MB in a Memory Profiler capture
Network	No new per frame or per keystroke request, and no more than two requests to open the board
Security related functions	A forged insert and a forged update are still refused, and the token table is still unreadable
Version number	Raised when the application changed, with the Android version code raised alongside it
Source reference	Recorded by the build itself in `BUILD_INFO.txt`, including whether the working tree was clean
Packaged build	Smoke tested on a device, not in the editor
Known issues	Written down, in the release notes and in the report, including the ones nobody has fixed

Primary Performance Metric to Monitor: the board fetch server leg, which is the time between the request going out on an established connection and the reply arriving, currently 91.1 ms of which about 43 ms is travel.

Reason: it is the one number that separates this project's work from the network's. The handshake and the distance are fixed costs the group does not control, and watching the total hides a growing query behind them. If TCP stays at about 43 ms and TLS at about 85 ms while the server leg climbs, the cause is the table, the query or the row count, which are the three things the group can actually act on.

When It Should Be Checked: Before Release, as a step in the release process, and additionally after any major feature that touches the database or the leaderboard code.

Part IX. Individual Contribution Record

This section is individual. The record below is Lacida's; Daquis and Manzanero complete their own against the same headings.

Item	Response
Student Name	Lacida, Kylo Bryan
Group Number	33
Area Worked On	Network, CPU, Memory and Testing. Build: none, for the reason recorded in Part VII
Component / File / Asset	`Tools/measure_backend.py` (new), `Tools/check_backend.py` (new), `Tools/perf_probe/` (new), `Docs/perf/` (seven evidence files), `Docs/Activity10_Performance.md` (this report), `.gitignore`, `.gitattributes`, `Docs/RELEASE_PROCESS.md`, `README.md`. Measured without modification: `Assets/Scripts/LeaderboardClient.cs`, `SupabaseClient.cs`, `PronunciationChecker.cs`, `WordVariants.cs`, `supabase/migrations/20261112_leaderboard.sql`, `supabase/functions/submit-score/index.ts`
Specific Contribution	Wrote the three measurement tools, established the baseline for the leaderboard retrieval scenario over the live Supabase project with phase split timing, measured the voice judgement's per attempt cost and allocations by compiling the shipping sources directly, measured the audio footprint from the files themselves, ran the nine backend functional checks and the regression pass, analysed the results, and concluded with evidence that optimization is not currently necessary
Profiling / Testing Evidence	`Docs/perf/20261113_backend_baseline.csv`, `20261113_backend_recheck.csv`, `20261113_backend_functional_checks.txt`, `20261113_backend_functional_recheck.txt`, `20261113_voice_judgement_baseline.txt`, `20261113_voice_judgement_recheck.txt`, `20261113_audio_footprint.txt`
Commit / Changeset Reference	One commit on `main`, on top of `921c894`, carrying the three tools, the seven evidence files and this report. Measurements apply to the released commit `c035c8f`, tag `v0.1.0`, since the measured sources are byte identical between the two
Build Responsibility	None this cycle. No build was required, and the Android toolchain is not on this machine. Responsibility for the next build stays with the member who produced v0.1.0

What I learned about the relationship between functional testing, performance measurement, optimization, and succeeding builds

Measuring first changed what I thought the problem was. I expected the leaderboard to be slow and found that most of its quarter second is a TLS handshake and the distance to the region, with about 48 milliseconds of actual server work. The judgement I assumed was expensive costs six microseconds. So there was nothing to optimise, and saying so with numbers is a result. The functional checks mattered more than the timings: a forged update answered HTTP 204, and only reading the row back proved it had changed nothing. Testing tells you whether a thing works, measurement tells you whether it is worth changing, and a build is only justified when one of those two says the application itself has to change.

(approximately 120 words)